Glossary (abbreviations)

DDL/DML/DCL	Definition / Manipulation / Control Lang.
ER	Entity-Relationship (model)
FD/MVD/JD	Functional / Multi-valued / Join Dependency
CK/PK/FK/SK	Candidate / Primary / Foreign / Super Key
RA/DRC/TRC	Rel. Algebra / Domain & Tuple Rel. Calculus
CP	Cartesian Product
NJ/CP-JN	Natural Join / Cartesian-product join
ACID	Atomicity, Consistency, Isolation, Durability
2PL/S2PL/SS2PL	2-Phase Locking (strict, strong-strict)
PG/SG/WfG	Precedence- / Serializability- / Wait-for-Graph
SK	Search Key (index)
HB/HT	Block height (B^+tree) / Hash Table height
LS/BS	Linear / Binary Search
BAT	Bucket Address Table (ext. hashing)
NLJ/BNLJ/INLJ	(Block-/Indexed-) Nested Loop Join
SMJ/HJ	Sort-Merge / Hash Join
LRU/MRU	Least/Most Recently Used (buffer)
NF/BCNF	Normal Form / Boyce-Codd NF (1–4NF)
MVA	Multi-Valued Attribute
LHS/RHS	Left-/Right-Hand Side (of an FD)
CTE	Common Table Expression (WITH)
TA/Tx	Transaction (atomic unit of work)
CS	Conflict-Serializable (schedule)
RC/ACA	Recoverable / Avoid Cascading Aborts
WAL	Write-Ahead Log
TID/RID	Tuple-/Record-ID (points to tuple)
DBMS	Database Management System

1. ER Diagram

Goal: model real world → relational schema.
Symbols: Entity=rectangle; Weak entity=double rect; Attribute=oval; PK=underlined; Partial key=dashed underline; Relationship=diamond; MVA=double oval; Derived=dashed oval; ISA=triangle.
Cardinality (Chen): 1:1, 1:N, M:N.
Participation: total=double line (must have), partial=single (may have).
Weak entity: no own PK; PK=PK(owner)+partial key;
isA / Specialization:
- **disjoint**: sub-entities don't overlap. **overlapping**: can be in $\geq$2.
- **total** (=double line): every super is in some sub. **partial**: not necessarily.

ER → Relational Schema

(1) Regular Entity: relation = simple attr; Composite attr → flatten; MVA → separate rel.
(2) Weak Entity: relation = own attr + PK(owner) as FK; PK = PK(owner) ∪ partial key.
(3) Binary 1:1:
- 1 total: FK in total-side rel pointing to other.
- Both total: merge both entities into 1 rel.
- Neither total: new rel for relationship or FK + NULLs.

(4) Binary 1:N: FK on N-side referring to 1-side. No new relation.
(5) Binary M:N: new relation, PK = PK(E1) ∪ PK(E2) + relation attrs.
(6) N-ary: new rel; PK = all participating PKs; (if one part. is 1, that PK becomes FK only).
(7) MVA A **of entity** E**:** new rel(A, PK(E)); PK = both.
(8) isA mapping:
- *All-super-sub*: rel(super), rel(sub, super-PK + sub-attr). Works always.
- *Total-only sub*: rel(sub) for each sub w/ super-attr+sub-attr.
- *Disjoint super-type*: 1 rel super + type-attr + (optional) sub-attr.
- *Overlap/Dis super-flag*: super rel + Boolean flag per sub.

Example (school-library Ex01):
Person(ID,FN,LN,Age) →
Employee(ID,Status,HireDate) + Student(ID,City,Street,Apt). FK Employee.ID → Person.ID. Borrow(SID,BID,ResDate,RetDate); Book(BID,Title,PubID→Publisher); Writes(AID,BID); Copy(CDate,BID,CoverType) – weak entity.

2. Relational Algebra (RA)

RA Operations

$\sigma_\theta(r)$: select tuples where θ true. θ: $\wedge, \vee, \neg$...
$\pi_{A,B}(r)$: project columns (duplicate-elim!).
$\rho_{S(B,C)}(r)$: rename rel to S, attrs to B, C.
$\cup$ union, $\cap$ intersect, $-$ difference.

Joins

Cartesian	$r \times s$, all pairs
Theta	$r \bowtie_\theta s = \sigma_\theta(r \times s)$
Equi	theta with $A = B$
Natural	equijoin on all common attrs, drop dups
Left outer	$r \text{-}\bowtie s$: keep all r, NULL pad

Division ÷

$r(R) \div s(S)$ with $S \subseteq R$:
Rewrite: $r \div s = \pi_{R-S}(r) - \pi_{R-S}((\pi_{R-S}(r) \times s) - r)$.

Aggregation

$_{G_1,\ldots,G_n}\vartheta_{F_1(A_1),\ldots}(r)$:
group by G, apply $F \in \{\text{sum,avg,min,max,count}\}$.
Count(*) counts NULLs; others ignore NULLs.

Example (max):
longest river: $\pi_N(\text{river}) - \pi_N(\text{river} \bowtie_{L<L'} \rho_{(L')}(\pi_L(\text{river})))$.
Atoll names+area:
$\pi_{Name,Area}(\sigma_{Type=\text{'atoll'}}(island))$
GDP over 100 econ:
$\pi_{Name,GDB}(\sigma_{GDP<100}(\text{country} \bowtie_{Code=Country} \text{eco}))$
Same-name cities:
$\pi_{cry}(\sigma_{N1\neq N2}(\text{cty} \bowtie_{Country,N=N1} \rho_{(N1,Cry,..)}(\text{cty})))$

3. Domain Relational Calculus:

Atoms: $r(t_1,\ldots,t_n)$ (table predicate), $x\theta y$, $x\theta c$.
Connectives: $\wedge, \vee, \neg, \Rightarrow$. **Quantifiers:** $\exists, \forall$.
Underscore _ = anonymous (existentially quantified, don't care).
Equivalences (First-Order Predicate Logic):
- $\forall x\ P \equiv \neg\exists x\ \neg P$
- $A \Rightarrow B \equiv \neg A \vee B$
- $\neg(A \wedge B) \equiv \neg A \vee \neg B$ (De Morgan)
- $\neg\exists x\ P(x) \equiv \forall x\ \neg P(x)$

Continents area>9M:
$\{N | \exists A(\text{enc}(\_, N, \_, \_) \wedge A > 9000000)\}$
(sum needed: not pure DRC, use RA aggregation)

Deserts not in CH:
$\{N, A | \text{desert}(N, A, \_) \wedge \neg\text{geoDes}(N, \text{'CH'}, \_)\}$

Border-of-border (not direct):
$\{C, P | \text{country}(C, \_, \_, \_, \_, P) \wedge \neg\text{borders}(C, \text{'SK'}, \_) \wedge \exists C'(\text{borders}(C, C', \_) \wedge \text{borders}(C', \text{'SK'}, \_))\}$

Longest river:
$\{N, L | \text{river}(N, L, \_) \wedge \neg\exists L'(\text{river}(\_, L', \_) \wedge L' > L)\}$

All friends ski:
$\{UN \mid \exists U(u(U, UN, \_) \wedge \forall F(f(U, F) \Rightarrow \exists S(s(S, F, \_, \_, \text{'Ski'}))))\}$

Pair, asymmetric: add $n_1 < n_2$ to avoid (B,A) duplicate.
∀ via ¬∃ pattern (RA-translatable):
$\forall y(r(y) \Rightarrow s(y)) \equiv \neg\exists y(r(y) \wedge \neg s(y))$.

5. Compact ops table

Op	RA	SQL
Select	$\sigma_P(r)$	WHERE P
Project	$\pi_A(r)$	SELECT DISTINCT A
CP	$r \times s$	FROM r,s
Theta join	$r \bowtie_\theta s$	r JOIN s ON θ
Natural	$r \bowtie s$	r NATURAL JOIN s
LeftOJ	$r \text{-}\bowtie s$	LEFT OUTER JOIN
Union	$r \cup s$	UNION
Diff	$r - s$	EXCEPT
Inter	$r \cap s$	INTERSECT
Div	$r \div s$	WHERE NOT EXISTS(...EXCEPT...)
Group	$_G\vartheta_F(r)$	GROUP BY G
Rename	$\rho_S(r)$	AS S

6. Query Trees

Heuristic optimization rules (apply in order):
1. Break σ with $\wedge$ into cascaded $\sigma_1\sigma_2\ldots$
2. Push σ as far down as possible (closest to leaves).
3. First do $\sigma/\ \bowtie$ producing smallest relation.
4. Replace $\sigma_\theta(\cdot \times \cdot)$ with $\bowtie_\theta$.
5. Push π down (deconstruct, add inner π).
6. Identify pipeline-able subtrees.

7. SQL Data Definition Language.

```
-- define a table with constraints, FK actions, domain, view, assertion:
CREATE TABLE country(
  Code CHAR(4) PRIMARY KEY,
  Name VARCHAR(35) NOT NULL UNIQUE,
  Capital VARCHAR(35),
  Area NUMERIC CHECK(Area>0),
  Pop INT DEFAULT 0,
  FOREIGN KEY (Capital) REFERENCES city(Name)
    ON DELETE CASCADE ON UPDATE SET NULL,
  CONSTRAINT areaPos CHECK(Area>=0)
);
CREATE DOMAIN Dollars AS NUMERIC(12,2);
CREATE VIEW v(A,B) AS SELECT ...;
CREATE ASSERTION aname CHECK(NOT EXISTS(...));
DROP TABLE r CASCADE; ALTER TABLE r ADD COLUMN x INT;
```

Triggers:
```
-- run f() automatically after each row inserted into r:
CREATE FUNCTION f() RETURNS TRIGGER AS $$
BEGIN ... RETURN NEW; END; $$ LANGUAGE plpgsql;
CREATE TRIGGER t AFTER INSERT ON r
FOR EACH ROW EXECUTE PROCEDURE f();
```

8. SQL DML basics

```
-- general query / insert / update / delete skeleton:
SELECT [DISTINCT] a, COUNT(*) AS c
FROM r [AS x] JOIN s ON x.A=s.A
WHERE p AND q
GROUP BY a HAVING COUNT(*)>1
ORDER BY a ASC, c DESC
LIMIT 10 OFFSET 5;
INSERT INTO r(A,B) VALUES(1,'x');
UPDATE r SET A=A+1 WHERE B='x';
DELETE FROM r WHERE A IS NULL;
```

Aggregates: COUNT([DISTINCT]*), SUM,AVG,MIN,MAX. All except COUNT(*) ignore NULL.
NULL semantics: 3-valued logic (T/F/Unknown). x = NULL → unknown; use IS NULL. AND: min(T,U,F); OR: max.
Patterns: LIKE 'a_b%'-1 char, a%=a+. ILIKE case-insensitive (PG).
Sets: UNION [ALL], INTERSECT, EXCEPT. Same arity.

Subqueries

IN/NOT IN: set membership (returns empty if any NULL in subq).
EXISTS/NOT EXISTS: correlated, safe w/ NULL, returns T iff ≥ 1 row.
ANY/ALL: x > ALL(q) = max.
Correlated: subquery references outer alias; re-evaluated per row.

"All X" pattern (universal):
```
-- Find all a's that satisfy all requirements in s.:
SELECT a FROM r
WHERE NOT EXISTS (
  SELECT * FROM s
  WHERE NOT EXISTS (
    SELECT * FROM t
    WHERE t.a=r.a AND t.b=s.b));
```
Division equivalent:
```
-- a-values whose t-partners cover all of s.b:
SELECT a FROM r WHERE NOT EXISTS (
  SELECT b FROM s
  EXCEPT
  SELECT b FROM t WHERE t.a=r.a);
```

9. SQL Advanced

```
-- min birth year among gold medalists, per game:
WITH youngest(GID,AID,AYob) AS (
 SELECT GID,AID,MIN(AYob) FROM Result NATURAL JOIN Athlete
 WHERE Medal='Gold' GROUP BY GID
) SELECT * FROM youngest;

-- transitive closure: all ancestor->descendant pairs:
WITH RECURSIVE anc(p,c) AS (
  SELECT p,c FROM parent
  UNION
  SELECT a.p,p2.c FROM anc a JOIN parent p2 ON a.c=p2.p
) SELECT * FROM anc;

-- all rows attaining the maximum x (ties kept):
SELECT * FROM r WHERE x=(SELECT MAX(x) FROM r);

-- youngest gold athlete winning most golds per game (2024):
WITH g AS (SELECT GID,AID,AYob,COUNT(*) NrG
           FROM Result NATURAL JOIN Athlete
           WHERE Medal='Gold' GROUP BY GID,AID,AYob)
SELECT GYear,AName,AYob,NrG FROM g NATURAL JOIN Games NATURAL
       JOIN Athlete g2
WHERE NOT EXISTS (SELECT 1 FROM g g3 WHERE g3.GID=g.GID
  AND (g3.AYob>g.AYob OR (g3.AYob=g.AYob AND g3.NrG>g.NrG)));

-- view: max top speed per (user, athlete), name-sorted:
CREATE VIEW v(UName,UID,AName,AID,MaxSpeed) AS
SELECT UName,UID,AName,AID,MAX(TopSpeed)
FROM u NATURAL JOIN s NATURAL JOIN a NATURAL JOIN r
GROUP BY UID,AID,UName,AName
ORDER BY UName ASC, AName ASC;

-- athletes older than the average athlete age:
SELECT AName,AYob
FROM Athlete a
WHERE AYob < (
  SELECT AVG(AYob)
  FROM Athlete
);
```

Common pitfalls:

- PK in SELECT → no DISTINCT needed; non-PK → DISTINCT.
- Once GROUP BY: only grouped attrs + aggregates in SELECT.
- HAVING is for aggregates; WHERE is for row filter.
- COUNT(non-PK) skips NULL; COUNT(*) counts every row.
- LEFT JOIN + COUNT(col)=0 finds "has no" (no-match).

10. FDs & Normalization

FD $X \to Y$: equal X-values ⇒ equal Y-values.
Normalize: avoid insert/update/delete anomalies; reduce redundancy.

Armstrong axioms (sound & complete)

- **Reflexivity:** $Y \subseteq X \Rightarrow X \to Y$
- **Augmentation:** $X \to Y \Rightarrow XZ \to YZ$
- **Transitivity:** $X \to Y, Y \to Z \Rightarrow X \to Z$
- **Union:** $X \to Y, X \to Z \Rightarrow X \to YZ$;
- **Decomp:** $X \to YZ \Rightarrow X \to Y$
- **Pseudotrans:** $X \to Y, WY \to Z \Rightarrow WX \to Z$.

Attribute closure X^+

$X^+ := X$; while change: if $\exists(V \to W) \in F$ with $V \subseteq X^+$: $X^+ := X^+ \cup W$. $X \to Y$ derivable iff $Y \subseteq X^+$.

Candidate Key (CK)

X= superkey: $X^+ = R$.
S_R= Candidate Key - minimal SK (not shortest, but no subset still CK)
Find all CKs:
1. Attrs only on RHS → never in CK.
2. Attrs only on LHS → always in CK.
3. Attrs both sides or none → check.
4. Compute closures of size-1, size-2, ... combos.
5. Prune supersets of found CKs.

Prime attr: attr in some CK. Non-prime: not in any CK.

Minimal cover F_{min}

1. RHS to single attrs: $X \to YZ \Rightarrow X \to Y, X \to Z$.
2. RM extraneous LHS attr: $XA \to Y$ extr. if $X \to Y$ in F.
3. RM redundant FD: $X \to Y$ redund. if derivable w/o it.

Normal Forms (NFs) – compact

compute closure X^+ = Σ..Z for all ...

1NF	all attrs atomic (no MVA, no nested rel).
2NF	+ no *partial* FD: no non-prime attr depends on part of CK.
3NF	+ for each $X \to A$: trivial, or X superkey, or A prime.
BCNF	+ for each $X \to A$ (nontrivial): X superkey.
4NF	BCNF + for each nontrivial MVD $X \twoheadrightarrow Y$: X superkey.

Anomalies

- **Insert:** can't insert dept without patient (partial PK NULL).
- **Update:** change Dept's Manager: must update many tuples.
- **Delete:** delete last patient in Dept: lose Dept info.

Decomposition properties

Lossless join: $R = R_1 \cup R_2$ is lossless iff $(R_1 \cap R_2) \to (R_1 \setminus R_2)$ or $(R_1 \cap R_2) \to (R_2 \setminus R_1)$ holds:
The common attributes must uniquely determine one side, so joining the decomposed relations does not create spurious tuples.

Dependency preserving: $(F|_{R_1}) \cup \ldots \cup (F|_{R_n})^+ = F^+$
where $(F|_{R_i})$ are all implied FDs for R_I:
All original functional dependencies can still be checked without recombining the decomposed relations.

Decomposition algorithms

BCNF decomp (lossless, may lose deps):
While some $X \to Y$ violates BCNF: split into $R_1(X \cup Y)$ and $R_2(R \setminus Y)$. Repeat.
3NF synthesis (lossless + dep-pres):
(1) Compute F_{min}. (2) For each FD $X \to Y$ in F_{min} make rel $R_i = XY$. (3) If no CK appears: add rel of CK. (4) Drop subset rels.
2NF decomp: for each FD $X \to A$ violating (X part of CK, A non-prime): $R_1(X \cup A)$, $R_2 = R \setminus A$.

Dependency-preservation example

$R(A,B,C,D)$, $F = \{B \to D, AD \to C, AC \to D, CD \to B\}$. Decomp $R_1(BD)$, $R_2(ABC)$.
$F|_{R_1} = \{B \to D\}$; $F|_{R_2} = \{AC \to ABC\}$ only (no nontriv on R_2).
$(F_1 \cup F_2)^+$: AD⁺=AD (not C!), CD⁺=CD (not B!). ⇒ **not dep-preserving**. AD→C, AD→B, CD→B lost.

Multivalued Dependencies (4NF)

Intuition: For each fixed value of X, the Y-values form a set that is independent of the rest of the attributes, forces a cross-product effect. Helps detect redundancies.
$student(SID, Course, Hobby)$ Leads to MVD:
$SID \twoheadrightarrow$ Course, $SID \twoheadrightarrow$ Hobby.
Add a new course → must duplicate it across all hobbies

$X \twoheadrightarrow Y$: for each X-value, set of Y-values independent of other attrs.
Trivial: $Y \subseteq X$ or $X \cup Y = R$.
Lossless 4NF decomp: $R_1(X \cup Y)$, $R_2(R \setminus Y)$.

Fullfill MVD's CD →→ B

r

A	B	C	D
'y'	'c'	1	2
'x'	'a'	1	3
'z'	'b'	2	2
'y'	'a'	1	2
'y'	'd'	2	2
'x'	'a'	1	2
'x'	'c'	1	2
'y'	'b'	2	2
'z'	'd'	2	2

11. Keys summary

Superkey	set X with $X^+ = R$ (uniquely determines)
Candidate K	minimal superkey (no proper subset is sk)
Primary K	one chosen CK (often NOT NULL+UNIQUE)
Alternate	non-PK candidate keys
Foreign K	attrs ref. PK of another rel
Prime attr	belongs to some CK
Composite	key with ≥ 2 attrs
Partial K	weak-entity key (qualified by owner)

12. NF transformation recipes

1NF→2NF	Eliminate partial FDs: R_1=FD attrs; R_2=rest+PK
2NF→3NF	Eliminate transitive non-prime deps: synth F_{min}
3NF→BCNF	For each viol. $X \to Y$: split $XY/R-Y$ (may lose dep)
BCNF→4NF	For nontriv MVD $X \twoheadrightarrow Y$ violating: $R_1 = XY, R_2 = R - Y$

Always check: lossless? dep-pres? higher NF? CKs of new rels?

13. Inference rules check

Tests whether a new dependency logically follows from the dependencies you already know. Use Armstrongs Axioms (sec. 10.1) to compare FDs!
$X \models Y$: $Y^+ \subseteq X^+$, every FD in Y derivable from X.
Intuition: all dependencies in Y logically follow from assumptions in X.
$X \equiv Y$: $X^+ = Y^+$.
Intuition: X and Y express the same information, even written diff.

14. Common FD pitfalls

- Add $\emptyset \to \emptyset$ trivially; trivial FDs $Y \subseteq X$ always hold.
- Check FDs against full closure F^+, not just listed FDs.
- If CK contains only 1 attr → rel is auto in 2NF (no partial possible).
- BCNF=less redundancy but may lose deps. 3NF=guaranteed dep-pres+lossless.
- Synthesis 3NF: minimal cover is essential to avoid extra rels.

18. Physical storage & indexes

Goal: fewest disk block accesses → cost = I/O time × #I/Os.
FileOrg: heap, sequential, hash, B^+tree-organized.

Index types

Primary (clustering)	on the order attr; file sorted by key; one per rel.
Secondary	on any attr; file not sorted by it; many per rel.
Dense	entry per tuple (= every search key value).
Sparse	entry per block (only ordered files).
Multi-level	index on index ($B+$tree natural).
Hash	equality only; bucket = hash(K).

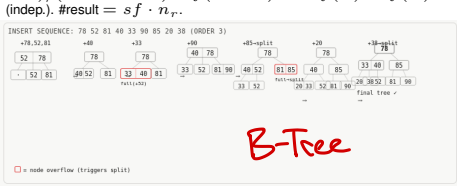
Index choice: ordered file → sparse primary; unordered → dense secondary; equality only → hash; range or both → B^+tree.

SELECT cost

A1 Linear scan: cost $= b_r$. On PK equi avg $b_r/2$.
A2 Binary search on ordered file: $\lceil \log_2 b_r \rceil$ + extra for non-unique.
A3 Primary index + equi on CK: $H_B + 1$ where $H_B = \lceil \log_{m/2} b_r \rceil$ B^+tree height.
A4 Primary idx + equi non-CK: $H_B + b/sf$ (SK blocks contiguous).
A5 Secondary idx + equi: $H_B + 1$ (CK); else $H_B + sf \cdot n_r$ (one block per tuple worst).
A6 Primary idx + range A≥X: $H_B + b_r \cdot sf$.
A7 Secondary idx + range: $H_B + sf \cdot n_r$ (each tuple = block access). Worst case linear scan A1 better.

Selectivity & estimation

$sf(A = c) = 1/V(A,r)$. $sf(A < c) \approx (c - \min)/(max - \min)$. $sf(A \wedge B) = sf(A) \cdot sf(B)$ (indep.). #result $= sf \cdot n_r$.



BAT sized 2^i where i=global prefix.
Insert value v, hash to bin, use last i bits as BAT index:
- If bucket has space: insert.
- Else if local prefix $j < i$: split bucket, redistribute.
- Else $j = i$: double BAT, $i \leftarrow i+1$, split.

Hash Index

Delete: remove; if buddy buckets both under capacity and same local prefix: coalesce (shrink BAT if all pairs merged).

20. Join algorithms

M=buffer blocks. Outer = r, inner = s. Aim outer = smaller.

Naive NLJ cost = $n_r \cdot b_s + b_r$. worst-case.

Block NLJ cost = $\lceil b_r/(M-2) \rceil \cdot b_s + b_r$. Outer should be smaller.

Memory-fit if $b_r \leq M-2$: cost = $b_r + b_s$.

INLJ for each tuple in r probe index on s. Cost = $b_r + n_r \cdot c$. c=index lookup. B$^+$tree: $H_B + 1$ (CK) / $+SK$blocks. Hash: $\approx 1-2$.

SMJ (sorted) cost = $b_r + b_s$. If unsorted: add sort cost (ext-sort).

Hash join needs hash on join attr; build small (s), probe r. Cost $\approx 3(b_r + b_s)$ if no in-mem; $b_r + b_s$ if fit.

Ex10 example: $|x| = 20000$ (20B/tuple, $\Rightarrow$ 204 tuples/block $\rightarrow b_x \approx 98$), $|y| = 50000$ (30B $\Rightarrow$ 136 tpb $\rightarrow b_y \approx 368$); $M = 200$.

- Both fit in M? $b_x + b_y = 466 > 200$ *no*. Only x fits.
- NLJ x outer: $20000 \cdot 368 + 98 = 7.36$M.
- BNLJ x outer, $M-2=198$ avail.: $\lceil 98/198 \rceil \cdot 368 + 98 = 466$.
- INLJ (hash on B): if B PK of y: $b_x + n_x \cdot 1 \approx 20098$.
- Merge join sorted: $b_x + b_y = 466$.

21. External sort-merge

Pass 0 read M blocks, sort in memory, write run. #runs = $\lceil b_r/M \rceil$.
Pass i: merge $M-1$ runs at a time. Passes = $\lceil \log_{M-1}(\lceil b_r/M \rceil) \rceil$.
Total Block-Transfers: $2b_r(\lceil \log_{M-1} \lceil b_r/M \rceil \rceil + 1)$.

2025 Task 9: $b_r = 327$, $M = 6$. Pass 0: $\lceil 327/6 \rceil = 55$ runs (54 of 6, 1 of 3). Pass 1: merge by 5 $\rightarrow \lceil 55/5 \rceil = 11$ runs of 30 (+1 of 27)$\rightarrow$12 runs. Pass 2: $\lceil 12/5 \rceil = 3$ runs of 150 (+1 of 27). Pass 3: 1 run of 327.
Ex10 (24 tuples, $M = 4$**):** Pass 0: 6 runs of 4. Pass 1 (merge 3): 2 runs of 12. Pass 2: 1 run of 24.

22. Optimization & query trees

Heuristics: push σ, π down; combine $\sigma \times \rightarrow \bowtie$; reorder joins; smallest first.
Cost-based: estimate #rows + I/O cost per plan; pick min.
2025 Task 4.2 rewrites observed:
1. Move $\sigma_{AID=11}$ down to leaf a before $\bowtie$.
2. Replace $\sigma_{UID=UID2}(u \times s)$ with $u \bowtie s$.
3. Push π_{UID} inside, $\pi_{UID,AID}$ before final $\bowtie$.
4. Execute joins producing smallest result first ($|a| < |u| < |s|$).

Pipelining: streaming tuples between operators (no materialization). Possible for σ, π chains; sort and hash-build break pipeline.

23. Storage & Buffer / EXPLAIN

Plan nodes: Seq/Index Scan, Bitmap Heap+Idx, Hash/Merge/Nested Loop Join. Disable: `SET enable_hashjoin=off;`. **Block (page)** = transfer unit (e.g. 4KB). **Buffer pool** caches; eviction: LRU, MRU.
TID/RID: (block#, slot). Used by indexes pointing to tuples.
Pinning: a buffer block must stay; unpinned only freeable.
Force/no-force: Must write to disk after Commit.
steal/no-steal: Allowed to write to disk before Commit.
Most: *steal/no-force* $\Rightarrow$ needs UNDO+REDO.

25. Transactions

TA: sequence of ops as atomic unit.
ACID: **A**tomicity (All or nothing), **C**onsistency (DB starts and stops in consistent state), **I**solation(Concurent TAs have no effect on each other), **D**urability (once TA finished, DBMS ensures that outcome would survive malfunction).

States

active $\rightarrow$ partially-committed $\rightarrow$ comm; or active $\rightarrow$ failed $\rightarrow$ aborted.

Conflicts

Ops by different TA on same item conflict if at least one is write:
- r-r: **not** conflict (swappable).
- r-w, w-r, w-w: conflict.
- different items: never conflict.

Serializability

Serial schedule: Transactions execute fully one after another, without interleaving.
Conflict-equivalent: Same operations, same output, different order. Can be obtained by swapping adjacent *non-conflicting* operations.
CS: equivalent to some serial schedule (no conflicting order).
Draw a precedence/serializability graph (PG = SG); if acyclic $\rightarrow$ schedule is CS.

PG:
- node per TA.
- edge $T_i \rightarrow T_j$ if T_i has op O_i, T_j later op O_j, they conflict, $i \neq j$.

Acyclic PG $\Leftrightarrow$ conflict-serializable. Topo-sort = a serial equivalent.
Steps to check:
1. Scan schedule, list conflicting pairs.
2. Build PG.
3. Check cycle. If acyclic $\rightarrow$ CS.

Recoverability classes

Reads-from: T_j reads from T_i if T_i writes X before T_j reads X.
RC: if T_j reads from T_i then c_i before c_j.
ACA (cascadeless): T_j reads from T_i only after c_i.
Strict: no read or write on item until last writer commits/aborts.
Hierarchy: Strict $\subset$ ACA $\subset$ RC.
Cascading rollback: when T_i aborts, all Tx that read its data also abort. Avoid via ACA.
2024 single-choice S:
$w_1(A)r_2(D)w_3(A)w_3(D)w_1(D)$
$c_1w_2(C)r_3(A)c_3c_2$.
- Conflicts: $w_1(A)$-$w_3(A)$, $r_2(D)$-$w_3(D)$, $w_3(D)$-$w_1(D)$, etc. PG has cycle $\Rightarrow$ **not CS**.
- T_2 reads nothing from T_3? $r_3(A)$ reads $w_1(A)$ but c_1 before $r_3(A) \rightarrow$ OK. $\Rightarrow$ **recoverable, cascadeless**.

Lock-based protocols

Lock types: S (shared, read), X (exclusive, write). Only S-S compatible.
2PL: every Tx has growing phase (only acquires) then shrinking (only releases). $\Rightarrow$ CS but *may cascade, may deadlock*.
S2PL: hold all X-locks until commit. $\Rightarrow$ recoverable + cascadeless.
SS2PL: hold ALL locks until commit. $\Rightarrow$ strict. (=S2PL)
2025 Task 6.1: $S = r_1(A), r_3(C), w_3(A), r_1(A), w_2(C)$. To do $w_3(A)$, T_3 needs X(A); but T_1 holds S(A) and reads A again later $\Rightarrow T_1$ would have to release S(A) before T_3's X(A), then reacquire $\Rightarrow$ **violates 2PL** (no reacquiring).

Deadlocks

WfG: node per Tx; edge $T_i \rightarrow T_j$ if T_i waits for lock held by T_j. If $T_i \rightarrow T_y \rightarrow T_j$ show $T_i \rightarrow T_j$
Cycle in WfG $\Leftrightarrow$ deadlock.
Recovery: pick victim (min cost: fewest writes, youngest), abort, roll-back. Min-cost = abort fewest.
Steps:
1. Trace each Tx in order, applying lock requests.
2. If lock conflicts: Tx waits, add WfG edge.
3. After last instruction, check WfG for cycles.
4. If cycle: deadlock; min-cost: abort ≥ 1 Tx per cycle.

26. Isolation levels (SQL)

Level	Dirty R	Nonrep R	Phantom
READ UNCOMMIT	may	may	may
READ COMMIT	no	may	may
REPEATABLE R	no	no	may
SERIALIZABLE	no	no	no

Anomalies in Synchronization

Dirty read: read uncommitted write. **Lost update:** two writes overwrite.
Nonrepeatable read: same query, different result, same Tx. **Phantom:** same predicate query, new rows appear.

27. Recovery

Log-based: WAL – log record written before data block flushed.
Log record types:
- $\langle T_i$ start$\rangle$
- $\langle T_i, X, V_{old}, V_{new} \rangle$ (UPDATE)
- $\langle T_i$ commit$\rangle$ / $\langle T_i$ abort$\rangle$
- $\langle$checkpoint $L\rangle$ where L=active Tx

Undo/Redo recovery: After crash, scan log:
- REDO list: Tx committed after last checkpoint.
- UNDO list: Tx started but not committed.
- Redo all (idempotent), then undo.

Checkpoint: flush log + dirty buffers, write checkpoint record. Reduces recovery scan.
Steal/no-force (typical): need both UNDO+REDO.
No-steal/no-force: no UNDO; **steal/force:** no REDO needed.

33. DCL / Security

`SELECT, INSERT, UPDATE, DELETE, ALL PRIVILEGES`).
Attribute-level only for `UPDATE/INSERT`. `public`=every user. `WITH GRANT OPTION`=grantee may re-grant. `RESTRICT`=error if revoke would cascade; default cascades.
Authorization graph: node per user, edge $A \rightarrow B$ if A granted to B. User has priv iff path from DBA. Revoke removes user + outgoing recipients no longer reachable from DBA.
Limits: only relation/view/attribute level (no tuple-level) $\Rightarrow$ use views for row filter; end-user IDs often missing in web apps.

```
-- combined example SQL commands:
GRANT SELECT ON student TO peter, paul, mary;
GRANT UPDATE(office_no) ON professor TO peter;  -- attr lvl
GRANT ALL PRIVILEGES ON r TO public;            -- everyone

CREATE ROLE instructor;                         -- roles
GRANT SELECT ON course TO instructor;
GRANT instructor TO john;                       -- role -> user
CREATE ROLE professor;
GRANT instructor TO professor;                  -- role -> role
GRANT professor TO sven;

GRANT SELECT ON student TO peter WITH GRANT OPTION;  --
      delegation
GRANT SELECT ON v TO bob GRANTED BY CURRENT_ROLE;    -- by role

REVOKE SELECT ON student FROM peter;                 -- cascades by
      default
REVOKE SELECT ON student FROM peter RESTRICT;        -- fail if
      cascade

-- row-level access via view:
CREATE VIEW csasst AS
  SELECT * FROM assistant WHERE area='computer science';
GRANT SELECT ON csasst TO bob;
REVOKE ALL PRIVILEGES ON assistant FROM bob;
```

EX. SQL: asymmetric pairs

"Pairs of (user, friend) who never had a valid session in same area."

```
-- (user,friend) pairs who never shared a session area:
SELECT u1.UName, u2.UName
FROM f
JOIN u u1 ON f.UID=u1.UID
JOIN u u2 ON f.FID=u2.UID
WHERE f.UID < f.FID AND NOT EXISTS (
  SELECT 1 FROM v um1
  JOIN v um2 ON um1.AID=um2.AID
  WHERE um1.UID=u1.UID AND um2.UID=u2.UID
);
```

Key tricks: $UID < FID$ avoids symmetric duplicates. NOT EXISTS = "never". View v already aggregates max speed per (user,area), so existence test on v = "had valid session".

EX. DRC translation

"Dates in Arosa when no participants snowboarded; consider sessions w/o runs." $\{D \mid \exists A(a(A, \text{Arosa}) \wedge s(\_, \_, A, D, \_) \wedge \neg\exists S(s(S, \_, A, D, \text{Snowboard})))\}$ **Insight:** need any session in Arosa on that date ($\Rightarrow$ second atom), plus no snowboard session ($\neg\exists$).

EX. RA from DRC

DRC: $\{UN \mid \exists U(u(U, UN, \_) \wedge \forall F(f(U, F) \Rightarrow \exists S(s(\_, F, \_, \_, \text{'Ski'}))))\}$
Translation (via $\neg\exists$):

$$\text{NS} \leftarrow \pi_{UID}(u) - \pi_{UID}(\sigma_{cat=\text{'Ski'}}(s))$$
$$\text{UwNSF} \leftarrow \pi_{UID}(f \bowtie_{FID=X} \rho_{(X)}(\text{NS}))$$
$$\text{UwoNSF} \leftarrow \pi_{UN}(u) - \pi_{UN}(u \bowtie \text{UwNSF})$$

"Users whose every friend skis" = u − (users with ≥ 1 non-skier friend).

EX. Query tree rewrite

Original: $\pi_{UN}(\sigma_{AID=11}(\sigma_{UID=UID2}(\rho_{(UID2,X,Y)}(u) \times s) \bowtie a))$.
Rewrites (since $|a| < |u| < |s|$):
- Push $\sigma_{AID=11}$ to leaf a.
- Combine $\rho(u) \times s + \sigma_{UID=UID2} \rightarrow$ natural join $a \bowtie s$ on AID first (smallest!).
- Project early: $\pi_{UID,AID}(s), \pi_{UID,UN}(u)$.

EX. Schedule example + MVD rules

S2 PG check (Ex11): scan conflicts (w-w/w-r/r-w only); build $T_i \rightarrow T_j$ edges; acyclic = CS.
MVD inference (Ex08): Complement $X \twoheadrightarrow Y \Rightarrow X \twoheadrightarrow R - XY$; Aug $X \twoheadrightarrow Y, V \subseteq W \Rightarrow XW \twoheadrightarrow YV$; Trans $X \twoheadrightarrow Y, Y \twoheadrightarrow Z \Rightarrow X \twoheadrightarrow Z - Y$; Replication $X \rightarrow Y \Rightarrow X \twoheadrightarrow Y$.

TIPP. Exam checklist + tips

DRC: bind every free var to a relation; _ don't-care; $\forall \equiv \neg\exists\neg$.
SQL: DISTINCT for non-PK; NULL-safe via NOT EXISTS; correlated subq for "all"; CTE.
FD/NF (biggest): (1) closure F^+, (2) find ALL CKs, (3) check 1-BCNF, (4) decompose, (5) check lossless+dep-pres.
Tx: conflict pairs $\rightarrow$ PG cycle; recoverable/cascadeless; 2PL violations; WfG cycle = deadlock.
Always: state assumptions; show closure/CKs; write cost formula then plug in; 2PL shrinking = no new locks.

TIPP. SQL anti-patterns

- `NOT IN` unsafe w/ NULL $\rightarrow$ use `NOT EXISTS`.
- `COUNT(col)` skips NULL; `COUNT(*)` doesn't.
- Aggregate in WHERE? No – use HAVING.
- "No match" = LEFT JOIN + IS NULL.
- Self-join: alias twice + $r_1.A < r_2.A$.
- UNION dedups; UNION ALL keeps dups.

TIPP. NF/CK fast finders

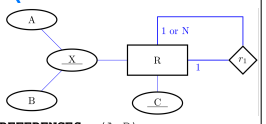
Find CKs: L=LHS-only attr always in CK; R=RHS-only never in CK; try $L \cup N$ first then add subsets of B (both-sides attrs).
NF shortcut: 2NF fails iff non-prime has partial dep on CK; 3NF fails iff $X \rightarrow A$ non-super non-prime; BCNF fails iff $X \rightarrow A$ non-super.

TIPP. Quick formula card

b_r	Number of blocks (pages) of relation r
n_r	Number of tuples (records) in relation r
sf	Selectivity factor (fraction of rows satisf. cond)
tpb	Tuples per block = $\lfloor B/\text{tuple size} \rfloor$
B	Block/page size in bytes
M	Number of available buffer pages in memory
$V(A,r)$	Number of distinct values of attribute A in relation r
c_S	Cost of one index lookup/search on relation S
b_r	$\lceil n_r/tpb \rceil$ blocks of rel
tpb	$\lfloor B/\text{tuple size} \rfloor$
H_B	$\lceil \log_{\lceil m/2 \rceil}(b_r) \rceil$ B$^+$tree height
INLJ cost	$b_r + n_r \cdot c_S$
BNLJ cost	$\lceil b_r/(M-2) \rceil \cdot b_s + b_r$
SMJ unsort cost	$b_r + b_s + \text{sort}(r) + \text{sort}(s)$
Ext-sort cost	$2b_r(\lceil \log_{M-1} \lceil b_r/M \rceil \rceil + 1)$
Init runs	$\lceil b_r/M \rceil$
Passes	$\lceil \log_{M-1}(\#\text{runs}) \rceil$
Selectivity equi	$1/V(A,r)$
$\lvert r \bowtie s \rvert$ est.	$\lvert r \rvert \cdot \lvert s \rvert / \max(V(A,r), V(A,s))$

Ex FOPL out of DDL:

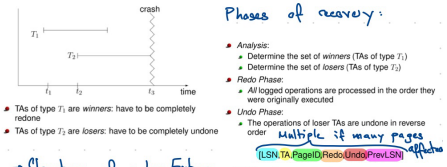
```
CREATE TABLE r(
  A INTEGER,
  B INTEGER,
  C INTEGER,
  D INTEGER,
  E INTEGER,
  PRIMARY KEY (A,B),
  FOREIGN KEY (D,E) REFERENCES r(A,B),
  UNIQUE (C),
  CHECK (D < A AND B < E)
);
```



CHECK as FOPL: $\forall A,B,D,E(\text{r}(A,B,\_,D,E) \Rightarrow (D < A \wedge B < E))$

PK & FK as FOPL:

$\forall A,B,C_1,D_1,E_1,C_2,D_2,E_2($
$(\text{r}(A,B,C_1,D_1,E_1) \wedge \text{r}(A,B,C_2,D_2,E_2)) \Rightarrow (C_1 = C_2 \wedge D_1 = D_2 \wedge E_1 = E_2)) \wedge$
$\forall D,E(\text{r}(\_,\_,\_,D,E) \Rightarrow \text{r}(D,E,\_,\_))$



Phases of recovery:
- Analysis:
  - Determine the set of winners (TAs of type T_1)
  - Determine the set of losers (TAs of type T_2)
- Redo Phase:
  - All logged operations are processed in the order they were originally executed
- Undo Phase:
  - The operations of loser TAs are undone in reverse order

Multiple if many pages affected

Structure of a Log Entry

- Redo:
  - Physical logging: after image
  - Logical logging: code that produces after image given before image
- Undo:
  - Physical logging: before image
  - Logical logging: code that produces before image given after image
- LSN (Log Sequence Number):
  - Every log entry has a unique identifier
  - LSNs have to be monotonically increasing
  - Determines the chronological order of the entries
- TA
  - ID of the transaction that has executed this operation

[LSN, TA, PageID, Redo, Undo, PrevLSN]

Example of Logfile:

A database system uses the ARIES recovery protocol. After a crash, the log and the disk contain the following information:

Log:

[#1, T_1, **BOT**, 0]
[#2, T_2, **BOT**, 0]
[#3, T_2, P_A, $A+=10$, #2]
[#4, T_3, **BOT**, 0]
[#5, T_2, **COMMIT**, #3]
[#6, T_1, P_A, $A-=5$, $A+=5$, #1]
[#7, T_3, P_B, $B*=2$, $B/=2$, #4]
[#8, T_1, P_B, $B+=7$, $B-=7$, #6]

Disk:

Page P_A : $A = 20$, LSN = 5
Page P_B : $B = 10$, LSN = 0

Undo/Redo Log: [LSN, TA, PageID, Redo, PrevLSN, UndoNextLSN]

6.1) [2 points] Show the result of the *Analysis* phase.
The analysis phase determines the following sets of winner and loser transactions:
- Set of winners: $\{T_2\}$
- Set of losers: $\{T_1, T_3\}$

6.2) [3 points] Show the state of the log and the disk after the successful execution of the *Redo* phase.

Log:
The log does not change during the redo phase and still contains entries #1 ... #8

Disk:
Page $P_A : A = 15$, LSN = 6
Page $P_B : B = 27$, LSN = 8

6.3) [5 points] Show the state of the log and the disk after the successful execution of the *Undo* phase.
These additions are made to the log:

Log:
[#8', T_1, P_B, $B-=7$, #8, #6)
[#7', T_3, P_B, $B/=2$, #7, #4)
[#6', T_1, P_A, $A+=5$, #8', #1)
[#4', T_3, -, -, #7', 0)
[#1', T_1, -, -, #6', 0)

Disk:
Page $P_A : A = 20$, LSN = 6'
Page $P_B : B = 10$, LSN = 7'

The database stores the first name, last name, and age of persons, who are identified by their ID. Every person is either an employee or a student (but not both). For the employees, the database stores their employment status and hiring date. The database stores the addresses of each student, with each address consisting of a city, a street, and an apartment number. Each employee may have a certificate, and many employees can have a same certificate. A certificate has a description, and is identified by its title. For a certificate, the database keeps track of the number of employees who have that certificate. Each student may borrow multiple books, and a book can be borrowed by multiple students. The reserve date and return date are stored upon borrowing. Books have their own ID through which they are identified, and a title. Every book is written by at least one author, and an author can write multiple books. Each author has a first name and last name, and is identified by an author ID. A book is published by exactly one publisher. Each publisher has a name, and is identified by its ID. A publisher publishes several books. Each book can also have multiple copies of itself. A copy of a book can only exist if the book is stored. Every copy of a book has a cover type, and is identified by a copy date and the ID of the book.